Students Database CRUD Application

Project report and demonstration guide
FastAPI / SQLite / Python

1. Objective

Build a backend application that creates, retrieves, updates and deletes student records. Preserve records between application restarts, validate incoming data and return useful error responses. Include an optional local AI extension without making it a dependency of the core application.

2. Delivered scope

Requirement	Implementation
Create, read, update, delete	POST, GET, PUT, PATCH and DELETE routes
Student information	Generated ID, name, birth date, contact and optional academic fields
Persistent storage	SQLite file with transactional writes
Testing and reproducibility	43 automated tests, live HTTP check and isolated demo
Submission package	Source, pinned dependencies, README, Postman collection and report
Optional extension	Restricted database questions; optional Ollama explanation

Project type

Individual educational backend project. The interactive Swagger API at /docs is the demonstration interface. All included sample records are fictional. No GitHub repository or external submission is claimed by this report.

3. Architecture and request flow

An HTTP client sends a request to FastAPI. Pydantic validates the request, the route calls parameterized database functions, and SQLite commits a successful write. FastAPI serializes the result into the documented response schema.

Layer	Responsibility
app/main.py	API routes, application lifespan, database path and HTTP responses
app/schemas.py	Input/output models, contact rules and date validation
app/database.py	Schema creation, bound SQL parameters and transactions
app/ai.py	Read-only question handling and optional local model call
tests/ and scripts/	Regression tests, sample data loader and repeatable demo

4. Data model

Column	SQLite type	Rule
student_id	INTEGER	Primary key; generated; not editable
name	TEXT	Required; 2-100 characters
date_of_birth	TEXT	ISO date; 1900-01-01 through today
email	TEXT	Optional, valid email, case-insensitive unique
phone	TEXT	Optional; 7-15 digits and allowed formatting
course	TEXT	Optional; maximum 100 characters
address	TEXT	Optional; maximum 300 characters
enrollment_date	TEXT	Optional ISO date; not before birth date

At least one contact field is required. SQLite enforces uniqueness and non-null constraints; the API enforces semantic validation. Dates use ISO text so ordered date comparisons are consistent.

5. Endpoints and behaviour

Method / route	Result
POST /students	201 Created with generated student ID and Location header
GET /students	200 paginated list with total, limit, offset and items
GET /students/{student_id}	200 record or 404 when absent
PUT /students/{student_id}	200 full replacement; omitted optional fields cleared
PATCH /students/{student_id}	200 merged update; validate the complete resulting record
DELETE /students/{student_id}	204 with an empty body; 404 when absent
POST /ai/ask	200 database-grounded answer for supported questions
GET /health	200 service/database readiness check

Search and validation

Combine name substring, exact course/email, and exclusive birth-date bounds. Search is case-insensitive for standard ASCII text; pagination allows 1-100 records per page with a nonnegative offset. Records are ordered by student ID. Invalid input returns 422, duplicate email returns 409, and missing records return 404.

Data integrity

All values are bound SQL parameters. Optional null values are supported, but PATCH cannot remove the only contact field. PATCH starts a write transaction before reading and merging, preventing competing partial updates from overwriting each other. Database operations commit on success and roll back on failure.

Sample create body

```
{
  "name": "Demo Student",
  "date_of_birth": "2006-02-10",
  "email": "demo@example.com",
  "course": "CSE"
}
```

6. Verification results

The project was executed with Python 3.12 in the build environment. All 43 automated tests passed. Separate live HTTP checks verified sample loading, repeat loading, server-process restart persistence and record operations. The raw evidence is included in docs/.

Test group	Verified behaviour
CRUD and HTTP contract	Create, read, PUT, PATCH, DELETE, Location header and empty 204
Input and missing data	Bad dates, empty names, contacts, unknown fields and missing IDs
Consistency	Duplicate emails, rollback, contact removal and optional nulls
Queries	Filters, pagination, exclusive date bounds and SQL-like input
Persistence and concurrency	Records survive app restart; parallel inserts produce unique IDs
Optional extension	Database answers, unsupported questions and mocked Ollama paths
Live HTTP	10 samples; second load skips 10; restart keeps 10; CRUD works

Evidence files

test_results.txt: actual pytest output.
demo_output.txt: actual in-process TestClient responses.
live_http_results.json: actual Uvicorn HTTP and process-restart checks.

Verification boundaries

Windows commands are supplied for the user, but execution was verified on Linux/Python 3.12. The pinned stack emits a non-failing upstream Starlette TestClient deprecation warning. A real Ollama model was not run; model communication was tested with mocked success and failure responses. No fabricated runtime screenshots are included.

7. Optional AI design

Supported forms are "How many students are there?" / "Count students" and "List students born after 2005." The year form means after the end of the stated year. Backend code retrieves facts with fixed read-only queries; the model never creates or executes SQL. The authoritative answer and records come from SQLite. An optional separate model explanation receives only aggregate facts. Unsupported questions are rejected.

8. Installation and demonstration

Extract the ZIP and open a terminal in students-database-crud. On Windows, install Python 3.12, then run:

```
py -3.12 -m venv .venv
.venv\Scripts\python.exe -m pip install -r requirements-lock.txt
.venv\Scripts\python.exe -m uvicorn app.main:app
```

Open **http://127.0.0.1:8000/docs**. Use a second terminal to run **.venv\Scripts\python.exe -m scripts.seed**. Demonstrate create, retrieve, filter, update and delete using Swagger or the included Postman collection. Use the actual returned student ID. Restart the server and retrieve a remaining record.

9. Requirement checklist

Assignment item	Location / evidence
Python, FastAPI and SQLite	app/ and requirements files
All four CRUD operations	app/main.py and tests/test_api.py
Unique ID, name, DOB and contact	app/schemas.py and app/database.py
Filter and retrieve profiles	GET /students and GET /students/{student_id}
Input/missing-record handling	422 / 409 / 404 paths and regression tests
README, installation and API testing	README.md; examples/Postman collection
Dummy data; no committed secrets	examples/students.json and .gitignore
GitHub link submission	User uploads final source to their chosen repository

10. Outcome and future work

The delivered application fulfills the mandatory local backend scope and includes a reproducible demonstration. Future extensions could add authentication, roles, migration tooling and a frontend. Authentication is not implemented, so use localhost and fictional data for this educational version.

References: FastAPI documentation (fastapi.tiangolo.com), Python sqlite3 documentation (docs.python.org/3/library/sqlite3.html), and Ollama API documentation (github.com/ollama/ollama/blob/main/docs/api.md).